

PROPOSAL AND IMPLEMENTATION PLAN FOR GRADUATION THESIS

Semester: 20251... | **University:** Hanoi University of Science and Technology (HUST) |
School: School of Information and Communication Technology (SoICT)

1. GENERAL INFORMATION

- Student's full name: Lê Văn Hậu (Student ID: 20226038)
- Class: ICT-02 (Batch: K67)
- Supervisor: Nguyễn Kiên Hiếu
- Thesis Title: Developing a Community Academic Knowledge Digitization and Management System Integrated with AI for Student Learning Support.

2. THESIS OBJECTIVES AND SCOPE

2.1 Objectives

- Develop an Academic Knowledge Infrastructure system to support students in contributing, standardizing, and utilizing community learning materials (PDFs, slides, photos).
- Develop an Agent Pipeline to automatically process uploaded documents, including: content extraction (OCR / Document Parsing) and text content normalization.
- Operate an Automatic Agent with Human-in-the-Loop (HITL) model with multi-level authorization: the Agent automatically assesses document quality; the Subject Reviewer and Admin verify and make the final decision.
- Develop AI Skills to support learning:
 - Virtual Tutor (AI Tutor) using RAG with a source citation mechanism
 - Automatic Mindmap generation from learning material content
 - Automatic Mock Test system generating review questions from materials
- Establish an AI Evaluation Framework combining automatic metrics (RAGAS) and Author Audit to evaluate the quality of answers and mitigate hallucination.

2.2 Thesis Scope

Target Audience

Students at the School of Information and Communication Technology - Hanoi University of Science and Technology.

Data Sources and Collection Potential

a) Evaluation Dataset (sample data):

Actual data collected by the thesis author and a group of friends during their studies at SoICT - HUST, including: lecture slides (PPTX/PDF), past exam papers, self-compiled review materials, and personal notes (photos, scanned PDFs) related to programming, algorithms, computer networks, operating systems, and databases.

High collection potential because:

- Accumulated over 3-4 years of study, ensuring authenticity and format diversity.
- Clear ownership (self-compiled or shared with friends' consent).
- Covers enough challenging formats (scanned PDFs, handwritten photos) for comprehensive OCR pipeline testing.

Important Note: The AI features in the system do not require fine-tuning. The collected materials serve two roles: (1) content for the knowledge base for system operation, and (2) an evaluation dataset to measure AI effectiveness.

b) Community Data:

Learning materials uploaded by students during the pilot study operation, used to test the entire flow from upload to the Document Evaluation Pipeline.

Data and Copyright Policy

- Users must confirm they have the right to share the document for academic purposes when uploading.
- The materials are solely for internal academic activities, not for commercial purposes.

Design Assumption on Document Classification

The system is built on the assumption that each document belongs to a specific course. This assumption is consistent with the actual organization of learning materials at Hanoi University of Science and Technology: courses have separate codes, and students store and share materials by specific course. **The Admin pre-populates the database with the official list of courses (sourced from SolCT's public and stable curriculum data), using the course code (e.g., IT3040, IT4062) as the primary key. When uploading, users are restricted to searching and selecting a course from a fixed dropdown list and cannot enter custom course names.** Therefore, the entire Document Evaluation Pipeline, domain routing, and HITL assignment are reliably based on this standardized course information. This is a deliberate design decision to ensure data quality and simplify the processing flow within the scope of the thesis.

2.3 Proposed Technologies

- **Programming Languages:** Backend - Python; Frontend - React / Next.js
- **AI / LLM:** Gemini 1.5 Flash (Google AI Studio, free tier ~1500 req/day); Llama 3 and Mistral via Groq API.
- **OCR Pipeline:** Primary Layer - Gemini Vision (OCR + one-step normalization); Backup Layer - Google Cloud Vision API. Post-extraction: normalization → chunking → embedding → Vector DB. Pipeline using LangChain/LlamaIndex.
- **Database:** Supabase (PostgreSQL + pgvector + Supabase Auth).
- **Message Queue:** Redis + BullMQ for OCR, embedding, Document Evaluation.
- **Internet Search:** Tavily Search API or SerpAPI (used in the Document Evaluation Pipeline).
- **CI/CD:** GitHub Actions; Frontend - Vercel; Backend - Render.

3. DETAILED IMPLEMENTATION PLAN (15 WEEKS)

Week	Task Content	Expected Result	MS
------	--------------	-----------------	----

1-2	Conduct current situation survey; collect initial sample dataset (group's university study materials); research OCR and Vector DB	Survey Report; dataset v0.1	
3	Develop outline; design Agent Pipeline and Document Evaluation Pipeline data flow; build Course Seed Documents for experimental courses	Outline + pipeline diagram + Course Seed	MS1
4	System requirements analysis; define AI Skills, metadata schema, scoring rubric for Agent 4, and the HITL authorization model	SRS Document + rubric v1	
5	System architecture and database design; design HITL workflow by domain	Architecture + ERD + HITL flow	
6	Finalize sample dataset; build Evaluation Dataset (50-100 manual Q&A pairs) for RAGAS and Author Audit	Dataset v1.0 + Eval dataset + design report	MS2
7	Build OCR and content extraction module (prioritize Gemini Vision Pipeline)	Prototype OCR pipeline	

8	Build document processing pipeline (normalization, chunking, embedding) integrated with Message Queue	Prototype ingestion pipeline	
9	Build Core RAG with source citation; integrate 4-agent Document Evaluation Pipeline (including Cold Start handling and Scoring Rubric)	Demo AI Tutor + Document Eval v1	MS3
10	Develop AI Skill: automatic Mindmap generation	Demo Mindmap	
11	Develop AI Skill: Mock Test Engine	Demo Mock Test	
12	System-wide integration; build HITL dashboard; optimize RAG; run Author Audit round 1 on Eval Dataset (20-30 questions)	Beta System + Audit Report v1	
13	Pilot study (5-10 students); evaluate RAGAS metrics; compare with Author Audit; collect UX feedback	RAGAS + Audit + UX Data Table	MS4
14	Finalize thesis report based on experimental results; system refinement	Complete Thesis Report	
15	Prepare slides, demo video, and plagiarism check	Complete Thesis	MS5

4. EXPERIMENT DESIGN AND SYSTEM EVALUATION

4.1 Target Audience for Experiment

Approximately 5-10 students at the School of Information and Communication Technology will participate in the pilot study.

4.2 AI Evaluation Framework

The system employs a two-tiered evaluation strategy: automatic metrics (RAGAS) and Author Audit. The two chosen frameworks are RAGAS (primary tier) and DeepEval (supplementary tier).

- **RAGAS:** Standardized metrics for RAG systems, natively integrated with the LangChain pipeline, using LLM-as-judge for automatic scoring.
- **DeepEval:** Supplements the ability to write unit tests for specific error types and integrates CI/CD, suitable for automating evaluation during development.

Tier 1 - Automatic Metrics (RAGAS)

Run on the entire Evaluation Dataset (50-100 Q&A pairs). Metrics set:

- **Faithfulness:** Is every claim in the answer supported by the retrieved context? This is the primary metric for measuring hallucination.
- **Answer Relevancy:** Is the answer appropriate and complete for the question?
- **Context Precision & Recall:** Quality of the retrieval step—are the retrieved passages truly relevant and sufficient to answer the question?
- **Citation Accuracy (self-developed):** Does the returned citation source match the actual context used to generate the answer?

Tier 2 - Author Audit (Human Evaluation on a small subset)

Purpose: Not to replace RAGAS, but to check calibration—confirming if RAGAS truly reflects user-perceived quality, and detecting subtle errors missed by RAGAS.

Scale: 20-30 questions randomly selected with stratification from the Evaluation Dataset (including easy, medium, and content-difficult questions).

Executor: The thesis author acts as the auditor. To mitigate confirmation bias, the audit is conducted blind (without viewing the RAGAS score first), and results are compared afterward.

Annotation form: For each question, the auditor assesses 3 dimensions:

- **Factual Correctness:** Is the answer content factually correct? (Correct / Incorrect / Partial)
- **Hallucination Flag:** Is there fabricated information not present in the original document? (Yes / No / Unsure)
- **Citation Valid:** Does the returned citation source actually contain the information used to answer? (Valid / Invalid / Irrelevant)

Method for Comparing Results: Calculate the Pearson correlation between the RAGAS Faithfulness score and the Factual Correctness of the Author Audit. If the correlation is low (< 0.6), analyze the cause and note it as a limitation. If it is high (> 0.6), use this result to reinforce the credibility of RAGAS in the report.

4.3 Comparison Baseline

Compare two configurations: (1) LLM without RAG, and (2) LLM with RAG using the academic knowledge base. RAGAS and Author Audit results are reported for both configurations to quantify the improvement level.

4.4 User Experience Evaluation

Students participating in the pilot study evaluate using a 1-5 Likert scale: ease of use, usefulness of the answers, quality of the mind maps, and quality of the mock test questions.

5. DOCUMENT EVALUATION PIPELINE

A core component ensuring the quality of the knowledge base. The entire pipeline runs asynchronously via Message Queue and integrates a self-correction mechanism.

5.1 Overview of 4-Agent Architecture

When a document is uploaded, the system creates a Document Evaluation Job and pushes it to Redis + BullMQ. A Worker processes 4 agents sequentially:

Agent 1 - Document Summarizer

- Receives OCR'd and normalized document content. Task: summarize document content according to a standardized structure including fields: main topic, list of concepts/skills mentioned, language, format, and estimated OCR quality. The output is a standardized JSON. The Agent has a retry mechanism if the output is not schema compliant (maximum 3 times).

Agent 2 - Course Context Aggregator

- Receives course information from document metadata (course name, course code). Task: aggregate two sources of context information:
 - Course description and official syllabus saved in the system (from PostgreSQL).
 - List and statistics of documents already accepted for that course (from the Academic Document Base), aiming to detect duplicate content.
- Output: JSON containing `course_description`, `existing_topics_coverage`, and `duplicate_flag`.

Cold Start Handling: When a course does not yet have a Course Seed Document, Agent 2 returns empty context with the flag **cold_start: true**. Agent 4 receiving this flag will automatically: (a) downgrade the confidence score to MEDIUM, (b) explicitly state the reason in the Evaluation Report, and (c) change the default Recommendation to NEEDS_REVIEW instead of APPROVE - forcing manual review. **Since the system uses an Admin-controlled, pre-populated list of official courses, the risk of cold start due to user input error is eliminated.** The Admin must ensure a minimum Course Seed Document (course name, short description, list of topics) is present for a course before its documents can be fully processed

Agent 3 - Internet Search Agent

- Based on the course name and university, this agent performs internet searches to gather reference information about the course from authoritative sources (department website, public syllabus, OpenCourseWare). Goal: supplement objective external context, helping the evaluation agent not be limited by internal data. Integration via Tavily Search API or SerpAPI. The Agent has a 15-second timeout and a graceful fallback if no results are found.

Agent 4 - Quality Evaluation Agent

Receives all context from the 3 preceding agents and performs evaluation according to a fixed Scoring Rubric hardcoded in the system prompt:

Dimension	Score	Criterion Description (hardcoded in system prompt)
Relevance	9-10 ▾	Document fully adheres to the course syllabus, most content belongs to the syllabus
	6-8 ▾	Most content is relevant, some parts are outside the scope but do not affect overall value
	3-5 ▾	Partially relevant, much content is outside the scope or does not match the course
	0-2 ▾	Not relevant or completely the wrong course
Completeness	9-10 ▾	Fully covers or exceeds the requirements of the Course Seed / syllabus
	6-8 ▾	Covers most main topics, missing a few minor points
	3-5 ▾	Covers only some topics, missing a lot of important content
	0-2 ▾	Too short, too superficial, or quality cannot be assessed due to low OCR quality
Quality	9-10 ▾	Clear presentation, logical structure, easy to read, no

		serious errors
	6-8 ▾	Decent presentation, may have some minor formatting or structural errors
	3-5 ▾	Poor presentation, many formatting errors, difficult to follow
	0-2 ▾	Unusable (OCR junk, completely chaotic content)

Output of Agent 4:

- Scores across 3 dimensions (0-10 each) according to the rubric above.
- Weighted Score = $0.4 \times \text{Relevance} + 0.3 \times \text{Completeness} + 0.3 \times \text{Quality}$.
- Duplicate Flag: detects duplication with documents already in the repository.
- **Recommendation:** APPROVE (Weighted ≥ 7.0 and no cold start) / NEEDS_REVIEW (5.0-6.9 or cold_start flag) / REJECT (< 5.0 or Duplicate = true).
- Full Evaluation Report with detailed reasons, saved to PostgreSQL and displayed on the HITL Dashboard.

5.2 Self-Correction Loop

Each agent checks the output schema after each run. If the output is invalid (incorrect JSON format, missing mandatory fields): the agent retries with an additional prompt describing the specific error (maximum 3 times). If it fails after 3 attempts: the job is marked EVALUATION_FAILED, transferred directly to the Admin for manual processing. All retry logs are recorded for debugging and prompt improvement.

5.3 Message Queue Integration

The pipeline operates entirely asynchronously via Redis + BullMQ. Job status is updated in realtime: QUEUED → PROCESSING → EVALUATED → APPROVED / REJECTED. Multiple workers run in parallel, ensuring no blocking when many documents are uploaded simultaneously.

6. HUMAN-IN-THE-LOOP (HITL) - DECENTRALIZED MODEL

The system applies a 3-level authorization model based on knowledge domain, which is practical and more scalable than an Admin-only model.

6.1 Role Hierarchy

Role	Description	Permissions
Student	Regular user	Upload documents; use AI Skills; view approved documents

Subject Reviewer	Senior student or teaching assistant assigned per course. During the pilot phase: handled by the thesis author's group.	View AI Evaluation Report; Approve / Reject documents within the assigned course domain; Add professional comments; Override AI Recommendation (with mandatory note)
Admin	System administrator	Full authority: manage Reviewers and domain assignment; override all decisions; approve documents outside the domain; input/update Course Seed Documents; configure the system; process EVALUATION_FAILED jobs

6.2 HITL Processing Flow

- After the Document Evaluation Pipeline is complete, the AI Evaluation Report is routed to the Subject Reviewer responsible for the corresponding course (based on domain mapping in the DB).
- APPROVE + Reviewer agreement → document is automatically entered into the Academic Document Base.
- NEEDS_REVIEW → Reviewer is required to manually review and make a decision.
- REJECT → Reviewer can override if there is a justifiable reason, along with a mandatory note.
- If there is no Reviewer for that course, or no response after 48 hours → automatically escalated to Admin.
- All decisions (Approve / Reject / Override) are fully logged, serving to improve Agent 4 prompts and the rubric later on.

6.3 HITL Dashboard Design

- **Reviewer Dashboard:** List of documents awaiting review by course; AI Evaluation Report with detailed score and reasons; Approve / Reject / Request More Info buttons; comment form.
- **Admin Dashboard:** System overview; manage Reviewers and domain assignment; process escalated documents and EVALUATION_FAILED jobs; manage Course Seed Documents; activity statistics.

7. KEY DIFFERENTIATING FEATURES OF THE THESIS

- **Systemic Approach:** Standardizing and restructuring community learning knowledge, transforming fragmented materials into an organized academic knowledge base.
- **Transparency:** All AI answers come with specific citations. All document approval

decisions include a full AI Evaluation Report and human decision log.

- **Automation:** The Agent Pipeline runs in the background via Message Queue, minimizing administrative effort and increasing scalability.
- **Two-Tiered Evaluation:** Combining RAGAS automatic metrics and Author Audit to achieve both scalability and manually verified reliability.

8. LIMITATIONS AND FUTURE DEVELOPMENT

8.1 Current Limitations

- Language: Optimized for Vietnamese and English.
- Formats: Unencrypted PDF, images (JPG/PNG), PPTX. Low-quality scanned PDFs may result in inaccurate OCR.
- Mathematical formulas: The ability to recognize and render LaTeX requires further verification.
- Experiment Scale: A pilot study of 5-10 students is insufficient for statistical conclusions; results are qualitative and illustrate feasibility.
- Author Audit single-annotator: Inter-annotator agreement cannot be calculated; it is framed as a 'manual calibration check' instead of full human evaluation.
- Subject Reviewer in pilot: Handled by the thesis author's group, no independent domain-specific Reviewers yet.
- Dependence on external services: Gemini API, Groq API, Supabase, Tavily/SerpAPI.
- Course Classification: The system operates based on an official course list predefined by the Admin. This is a design decision intended to simplify the document processing workflow. Limitations: Since some university courses may have different codes and titles despite sharing similar curricula, classification based strictly on specific course codes may be restrictive and could limit the potential for interdisciplinary content discovery.

8.2 Future Directions

- Expand support for mathematical formulas (MathOCR / Mathpix API).
- Increase experiment scale to 50+ students for statistical results.
- Expand Author Audit to multi-annotator with volunteer students, adding Cohen's Kappa calculation.
- Develop additional AI Skills: Automatic Summarization and Document Diff.
- Integrate RLHF-lite from user feedback to improve answer quality over time.
- Build a feedback loop from HITL override to the pipeline: save 'disagreement cases' (AI wrong, human correct) as data to improve Agent 4's rubric and prompt.
- Expand the target audience to the entire Hanoi University of Science and Technology.

9. COMMITMENT TO IMPLEMENTATION

I commit to implementing the plan as scheduled and regularly reporting the results to the Supervisor.

Hanoi, date month year 202...	
---	--

Implementing Student	Supervisor's Comments
(Sign and print full name)	(Sign and write comments)
Hậu	